Technical Design

Pipeline steps

1. **Load:** Read PDFs from a folder and candidate records from a CSV.
2. **Extract:** Pull candidate details from the PDFs using regex rules and/or AI.
3. **Lookup:** Match each resume to a CSV row using email and/or phone number.
4. **Fetch:** If a GitHub link exists, call the GitHub API to get profile data.
5. **Normalize:** Standardize formats (phone to E.164, location to ISO country codes, skills to lowercase, dates to MM-DD-YYYY).
6. **Consolidate:** Merge data from all sources and choose the "best" record by a completeness/quality score (skills, jobs, education). Track where each field came from.
7. **Filter:** Apply a runtime JSON config to output only requested fields and validate types.

Canonical schema / formats

1. Candidate_id (string)
2. full_name (string)
3. emails (list of strings)
4. phones (list of strings, E.164)
5. location (string, 2-letter ISO country code)
6. links (list of strings)
7. skills (list of lowercase strings)
8. experience (list of objects)
9. education (list of objects)
10. github_profile_data (object)
11. provenance (list of objects describing field origins)

Merge and conflict policy

1. **Match keys:** email and phone number.
2. **Winner:** the row with the higher quality score (based on counts of skills, jobs, and education entries).
3. **Confidence:** provenance is recorded for the final chosen values.

Custom output config (runtime JSON)

1. **Projection:** maps source keys to target keys.
2. **Validation:** checks output types.
3. **Missing data handling:** can insert null, omit the field, or fail with an error (based on config).

Edge cases

1. Invalid emails (e.g., malformed addresses) are ignored.
2. If there's no CSV match, output the candidate anyway, with candidate_id set to null.
3. If phone numbers lack a country prefix, use the CSV location to format them correctly.
4. No weird name matching is done due to time limits - only exact email/phone matches.